



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería del Software

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

MÁLAGA, March 19, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADUADO EN INGENIERÍA DE LA SALUD

Mención en Bioinformática

Biblioteca extensible de optimización metaheurística en Rust

**Extensible metaheuristic optimization library in
Rust**

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, MARCH 19, 2026

Fecha defensa: Septiembre de 2025

Resumen

El presente Trabajo de Fin de Grado tiene como objetivo principal el diseño y desarrollo de una biblioteca de optimización metaheurística de carácter extensible y modular, basada en una filosofía de cero dependencias y programada íntegramente en Rust, un lenguaje de programación compilado orientado a la seguridad de memoria y al alto rendimiento.

Palabras clave: Rust, Metaheurísticas, Optimización, Biblioteca

Abstract

Keywords: Rust, Metaheuristics, Optimization, Library

Agradecimientos

(quiero poner un agradecimiento, esto lo haré lo último

Resumen	1
Abstract	3
Agradecimientos	5
1 Introducción	13
1.1 Motivación	13
1.2 Objetivos	14
1.3 Tecnologías a utilizar	14
2 Metodología de trabajo	17
2.1 Modelo de desarrollo	17
2.2 Ecosistema y herramientas de desarrollo	17
2.3 Fases del proyecto	17
2.3.1 Investigación y capacitación	17
2.3.2 Definición de requisitos y planificación	18
2.3.3 Implementación técnica	18
3 Optimización y Metaheurísticas	21
3.1 El Problema de Optimización	21
3.2 Métodos Exactos frente a Métodos Aproximados	22
3.3 Heurísticas y Metaheurísticas	23
3.4 Clasificación de las Metaheurísticas	24
3.4.1 Metaheurísticas basadas en trayectoria	24
3.4.2 Metaheurísticas basadas en población	24
3.5 Extensión a la Optimización Multiobjetivo	24
4 Especificación y diseño	25
4.1 Arquitectura	25
4.2 Módulos	25
4.2.1 Algoritmos	26
4.2.2 Observadores	27
4.2.3 Operadores	28
4.2.4 Problemas	28

4.2.5	Conjuntos de soluciones	28
4.2.6	Soluciones	29
4.2.7	Experimentos	29
4.2.8	Utilidades	30
5	Implementación y pruebas	31
5.1	Implementación	31
5.2	Desarrollo de pruebas	33
5.2.1	Pruebas unitarias	33
5.2.2	Pruebas de integración	34
6	Validación experimental	37
7	Conclusiones y líneas futuras	41
	Apéndice A. Installation Manual	43
A.1	Requisitos	43
A.2	Instalación desde el repositorio	43
A.3	Instalación como <i>crate</i> mediante Cargo	44

Índice de Figuras

1.1	Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [8].	15
4.1	Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [10].	25

Nomenclatura

framework conjunto estandarizado de herramientas, librerías, reglas y buenas prácticas prediseñadas para agilizar el desarrollo de software

SCRUM marco de trabajo ágil para gestionar proyectos complejos

1

Introducción

1.1 Motivación

La optimización metaheurística constituye una herramienta fundamental para la resolución de problemas complejos en aquellos ámbitos donde los métodos exactos resultan inviables, ya sea debido al elevado coste computacional o a la naturaleza no lineal y no convexa de los espacios de búsqueda. Este tipo de técnicas ha demostrado su utilidad en una amplia variedad de aplicaciones, como la planificación y asignación de recursos, la optimización de rutas, el diseño de sistemas, la inteligencia artificial, el aprendizaje automático y la ingeniería en general.

En distintos lenguajes de programación existen bibliotecas maduras que facilitan la aplicación de técnicas metaheurísticas. Por ejemplo, *jMetal* es un framework ampliamente reconocido para la optimización multiobjetivo con metaheurísticas [6]. Su versión en Python, *jMetalPy*, ofrece un entorno extensible y orientado a objetos para experimentar con técnicas clásicas y avanzadas de optimización [4]. Asimismo, *MEALPY* (MEta-heuristic ALgorithms in PYthon) proporciona implementaciones de numerosos algoritmos inspirados en la naturaleza, tanto clásicos como híbridos, para abordar problemas complejos [11]. Sin embargo, dentro del ecosistema de Rust, el soporte para la optimización metaheurística es todavía limitado, lo que dificulta la adopción de estas metodologías en proyectos desarrollados en dicho lenguaje y pone de manifiesto la necesidad de disponer de herramientas nativas que se integren de forma natural con sus principios de diseño, especialmente en lo relativo al rendimiento, la seguridad de memoria y el control de recursos.

Este Trabajo de Fin de Grado surge con la motivación de dar respuesta a esta necesidad mediante el desarrollo de una biblioteca de optimización metaheurística modular y extensible. No obstante, abordar un proyecto de esta envergadura resulta especialmente desafiante en las etapas iniciales, particularmente en ausencia de experiencia previa. Afrontar un trabajo de fin de carrera implica

recopilar y consolidar los conocimientos adquiridos a lo largo de la titulación y aplicarlos en un proyecto real, lo cual supone una tarea de notable complejidad. Por ello, la elección de un tema que despierte interés, curiosidad y motivación personal resulta un factor clave para el desarrollo del proyecto.

El diseño y desarrollo de algoritmos metaheurísticos conforma un campo especialmente atractivo, al combinar fundamentos de la teoría de la computación, la optimización y la inteligencia artificial. Su aplicación en dominios tan diversos como la logística, la ingeniería o la biología computacional pone de manifiesto su versatilidad y su impacto en la resolución de problemas reales.

En el momento de inicio de este proyecto, no existían bibliotecas consolidadas orientadas al desarrollo de este tipo de soluciones en el lenguaje Rust, lo que constituyó una motivación adicional para su realización. Asimismo, el propio lenguaje Rust actuó como detonante del proyecto, al tratarse de una tecnología moderna, diseñada con un fuerte énfasis en la seguridad de memoria y el rendimiento, y especialmente adecuada tanto para el aprendizaje como para el desarrollo de software robusto y eficiente.

1.2 Objetivos

El objetivo principal de este proyecto es desarrollar una biblioteca en Rust que implemente algoritmos metaheurísticos para la optimización de problemas complejos. Debe ser capaz de resolver correctamente multitud de problemas de único objetivo y un conjunto pequeño de problemas multiobjetivo. Idealmente, esta biblioteca debería ser fácil de usar, eficiente y extensible, permitiendo a los usuarios adaptar los algoritmos a sus necesidades específicas. Para garantizar el cumplimiento de estas premisas, se ha realizado un desglose exhaustivo de los requisitos del sistema, donde se definen formalmente las funcionalidades y restricciones que guiarán el desarrollo de la herramienta.

1.3 Tecnologías a utilizar

Como lenguaje de programación, se ha elegido **Rust**, debido a su enfoque en el rendimiento, por su capacidad de manejo eficiente de la concurrencia y diferentes hilos, lo que lo hace ideal para la implementación de algoritmos heurísticos complejos que a menudo requieren un procesamiento paralelo intensivo. Además, su creciente ecosistema y su tipo de sistema robusto nos permiten construir soluciones de optimización escalables y de alto rendimiento. También, Rust ha comenzado a consolidarse como uno de los lenguajes de programación utilizados en el desarrollo del *kernel* de Linux, dejando atrás su carácter experimental y siendo aprobado oficialmente para la implementación de determinados componentes del núcleo, lo que refuerza su madurez, fiabilidad y adecuación para sistemas críticos [7].

Uno de los principales motivos por el que Rust es tan buena elección es por su gestor de paquetes y compilación, **Cargo**, que facilita la gestión de dependencias y la construcción de proyectos, permitiendo a los desarrolladores centrarse en la implementación de algoritmos en lugar de en la configuración del entorno.

Este gestor de paquetes se encuentra entre los más populares y admirados por los desarrolladores, como se puede observar en la estadística de la encuesta anual de Stack Overflow, donde Cargo destaca como uno de los gestores de paquetes más valorados para el desarrollo e infraestructura en la nube durante 2025 [8].

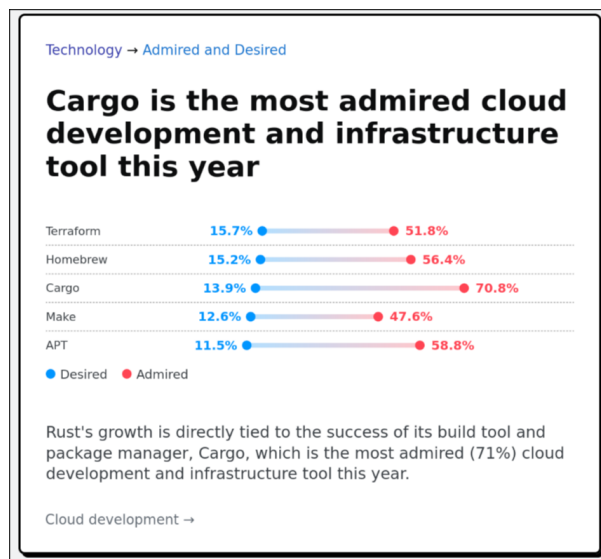


Figura 1.1. Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [8].

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover**, de **JetBrains**, que es un IDE específico para Rust. Esto fue debido a que en ese momento los conocimientos sobre Rust eran muy limitados y se necesitaba un entorno más guiado e invasivo para poder progresar.

Para la documentación y redacción del proyecto, se ha utilizado $\text{\LaTeX}$, para el control de versiones **Git** junto con **GitHub** como plataforma de alojamiento remoto.

2

Metodología de trabajo

En esta sección se describen los procedimientos, herramientas y fases seguidas para la consecución de los objetivos del proyecto.

2.1 Modelo de desarrollo

Para el desarrollo del proyecto se ha adoptado una metodología iterativa, inspirada en el marco de trabajo SCRUM, pero adaptada a las particularidades de un entorno de trabajo unipersonal. Esta adaptación permite conservar las ventajas de la planificación incremental y la revisión continua, ajustándolas a la naturaleza y alcance del proyecto.

Como herramienta de apoyo a la gestión y seguimiento de las tareas se ha utilizado JIRA, cuyo uso y configuración dentro del proyecto se detallarán con mayor profundidad en el apartado siguiente.

2.2 Ecosistema y herramientas de desarrollo

La selección de las herramientas de trabajo se realizó buscando maximizar la eficiencia en la detección de errores y la navegación por el código. Se utilizó mayoritariamente Visual Studio Code como entorno de desarrollo principal, aprovechando su versatilidad y el robusto soporte proporcionado por la extensión *rust-analyzer* [2].

2.3 Fases del proyecto

Un proyecto como el que nos encontramos debe estructurarse en varias partes, en este caso en tres etapas diferenciadas:

2.3.1 Investigación y capacitación

Debido a la pronunciada curva de aprendizaje de Rust, esta fase inicial se centró en la asimilación de sus fundamentos sintácticos y las particularidades de su compilador. Para consolidar estos conceptos,

se llevó a cabo la migración de un proyecto personal previo (una aplicación de escritorio), trasladando su arquitectura basada en Java y Javalin hacia un ecosistema nativo en Rust mediante el framework Tauri [9].

Paralelamente, se desarrolló un repositorio de pruebas a modo de entorno de experimentación, destinado a la implementación de pequeños módulos funcionales que permitieron dominar aún más el lenguaje. Esta etapa de formación técnica se complementó con un estudio exhaustivo sobre metaheurísticas y algoritmos genéticos, sentando las bases teóricas necesarias para el posterior diseño de la biblioteca.

Dentro del estudio técnico, cobró especial relevancia el análisis del repositorio y de la arquitectura de jMetal [6], un framework de optimización metaheurística multiobjetivo escrita en java por investigadores de la universidad de Málaga, ayudó a comprender los patrones de diseño y las estructuras de datos necesarias para implementar metaheurísticas de forma extensible.

2.3.2 Definición de requisitos y planificación

Durante esta etapa se realizó un trabajo de introspección, se analizaron los objetivos y se elaboró un documento donde se especificaron todos los requisitos del sistema.

A partir de estos requisitos, se creó un *backlog* en Jira, donde se registraron todas las tareas necesarias para completar el proyecto. Posteriormente, las tareas se organizaron en seis “sprints” y se clasificaron en las columnas “Por hacer”, “En curso”, “Por revisar” y “Listo”. Esta organización permitió un seguimiento claro del progreso y facilitó la comunicación con el tutor, mostrando periódicamente las tareas en la columna “Por revisar” para recibir su aprobación. Una vez que el tutor daba el visto bueno, las tareas se marcaban como “Listo”, asegurando un flujo de trabajo controlado y transparente [1].

2.3.3 Implementación técnica

Una vez definidos los requisitos del sistema y establecida la planificación del proyecto, se procedió a la implementación técnica de la biblioteca de optimización metaheurística. Esta fase tuvo como objetivo materializar las decisiones de diseño adoptadas previamente, dando lugar a un sistema funcional, modular y extensible, desarrollado íntegramente en el lenguaje Rust.

El proceso de implementación se llevó a cabo de forma incremental, permitiendo validar progresivamente las abstracciones definidas y adaptar la arquitectura a las particularidades del lenguaje y a los requisitos detectados durante el desarrollo.

La implementación estuvo fuertemente condicionada por las características propias del lenguaje Rust. La ausencia de herencia clásica y el uso intensivo de *traits* influyeron directamente en el diseño

de las abstracciones, fomentando un enfoque basado en la composición y el polimorfismo estático.

Asimismo, el sistema de propiedad y préstamos de Rust actuó como una herramienta de validación temprana, obligando a definir de forma explícita la gestión de memoria y el ciclo de vida de las estructuras de datos. Aunque este modelo supuso una dificultad inicial, permitió garantizar un alto nivel de seguridad y robustez en la implementación final.

La implementación de la biblioteca se realizó siguiendo un orden lógico que permitió construir el sistema de forma progresiva. En primer lugar, se desarrollaron las abstracciones básicas, tales como las representaciones de soluciones y problemas. Posteriormente, se implementaron los algoritmos de optimización, seguidos de los operadores fundamentales y de los mecanismos de observación.

Este enfoque permitió validar cada componente de manera aislada antes de integrarlo en el sistema global, reduciendo la complejidad del proceso de depuración y facilitando la detección de errores conceptuales en fases tempranas del desarrollo.

Durante la fase de implementación se llevó a cabo una validación continua del sistema, apoyándose principalmente en el compilador de Rust como herramienta de detección de errores y en la ejecución de pruebas y ejemplos controlados para verificar el comportamiento de los algoritmos.

Asimismo, se realizaron refactorizaciones periódicas del código con el objetivo de mejorar su claridad, reducir duplicidades y adaptar la arquitectura a nuevas necesidades detectadas durante el desarrollo. Este proceso iterativo permitió alcanzar una implementación estable y coherente con los objetivos iniciales del proyecto.

3

Optimización y Metaheurísticas

En numerosas disciplinas de la ingeniería, la economía y las ciencias de la computación, surge de manera recurrente la necesidad de tomar decisiones que maximicen el beneficio o minimicen el coste de un sistema dado. Este proceso de búsqueda de la mejor alternativa posible, dentro de un conjunto de soluciones factibles, es lo que se conoce formalmente como optimización. En este capítulo se introducen los conceptos fundamentales de la optimización matemática, las limitaciones de los métodos clásicos y el papel crucial que desempeñan las metaheurísticas en la resolución de problemas complejos.

3.1 El Problema de Optimización

De forma general, la optimización matemática trata de encontrar la mejor solución posible, evaluada según un criterio cuantitativo, de entre un conjunto de alternativas disponibles. Formalmente, un problema de optimización mono-objetivo se formula como la búsqueda de un vector de variables de decisión $x = (x_1, x_2, \dots, x_n)$ que optimice (ya sea minimizando o maximizando) una función objetivo escalar $f(x)$. Este problema se define matemáticamente de la siguiente manera:

$$\text{Minimizar } f(x)$$

$$\text{sujeto a } x \in \Omega$$

Sin pérdida de generalidad, en la literatura algorítmica suele trabajarse exclusivamente con problemas de minimización, dado que cualquier problema de maximización puede transformarse trivialmente

multiplicando la función objetivo por -1 (es decir, $\max f(x) = \min -f(x)$). Sin pérdida de generalidad, en la literatura algorítmica suele trabajarse exclusivamente con problemas de minimización, dado que cualquier problema de maximización puede transformarse trivialmente multiplicando la función objetivo por -1 (es decir, $\max f(x) = \min -f(x)$). El vector x representa las incógnitas del sistema sobre las cuales el proceso de resolución tiene control directo. La función objetivo $f : \Omega \rightarrow \mathbb{R}$ es la métrica que asigna un valor de calidad o coste a cada vector propuesto.

El conjunto Ω denota el espacio de búsqueda o la región factible del problema. Lejos de ser un espacio infinito o arbitrario, en los escenarios de ingeniería y del mundo real, este espacio está delimitado geométrica y matemáticamente por un conjunto de restricciones que el vector x debe cumplir de manera estricta. Estas restricciones imponen límites físicos, lógicos o económicos al sistema, y suelen expresarse mediante funciones de desigualdad ($g_i(x) \leq 0, i = 1, \dots, m$) y de igualdad ($h_j(x) = 0, j = 1, \dots, p$).

Por consiguiente, Ω está constituido única y exclusivamente por aquellos vectores que satisfacen simultáneamente todas estas condiciones. Si un algoritmo de optimización genera una solución que recae fuera de las fronteras de Ω , dicha solución se considera infactible. En el diseño de algoritmos y metaheurísticas, las soluciones infactibles suponen un desafío; por lo general, son descartadas inmediatamente o, de forma más sofisticada, se les aplica una penalización severa en su valor de $f(x)$ para guiar la búsqueda iterativa de vuelta hacia el interior de la región factible.

3.2 Métodos Exactos frente a Métodos Aproximados

Para abordar la resolución de los problemas matemáticos de optimización, existen tradicionalmente dos grandes familias de algoritmos: los métodos exactos y los métodos aproximados. La elección entre un enfoque u otro depende intrínsecamente de la naturaleza del problema, el tamaño de la instancia a resolver y los recursos computacionales disponibles.

Los métodos exactos exploran el espacio de búsqueda de una manera sistemática y exhaustiva, de tal forma que garantizan matemáticamente encontrar la solución óptima global del problema. Entre las técnicas más destacadas de esta familia se encuentran la Programación Lineal (como el algoritmo Simplex), la Programación Dinámica y los algoritmos de Ramificación y Acotación (*Branch and Bound*). Cuando el problema presenta un modelo matemático tratable y un tamaño de entrada moderado, estos métodos son, sin lugar a duda, la opción preferida debido a la certeza de sus resultados.

Sin embargo, su aplicabilidad se ve severamente limitada en la práctica industrial y de investigación. Muchos de los problemas de mayor interés y aplicabilidad en el mundo real pertenecen a las clases de complejidad NP-completos o NP-duros. Un ejemplo paradigmático es el Problema del Viajante de

Comercio (TSP, por sus siglas en inglés), donde se busca la ruta más corta que visite un conjunto de ciudades. En estos escenarios, el espacio de soluciones posibles crece de forma factorial o exponencial con respecto al tamaño del problema ($O(n!)$ o $O(2^n)$). Este fenómeno, conocido como "explosión combinatoria" o "la maldición de la dimensionalidad", provoca que un algoritmo exacto requiera tiempos de ejecución inasumibles (años o incluso siglos de cómputo) para instancias de tamaño realista.

Es precisamente ante esta intratabilidad computacional donde los métodos aproximados, concretamente los algoritmos heurísticos, cobran un protagonismo absoluto y se convierten en la única alternativa viable. La principal motivación de un heurístico es encontrar una solución en un tiempo razonable, especialmente para problemas NP-completos o de gran escala, donde un algoritmo exacto tardaría demasiado o sería inviable. Estos algoritmos logran esta eficiencia sacrificando la garantía de encontrar la solución óptima o perfecta.

A diferencia de los enfoques exactos, la naturaleza de los métodos aproximados es inherentemente incompleta, ya que no exploran todo el espacio de soluciones posibles. En su lugar, producen soluciones que son buenas o cercanas al óptimo (también conocidas como soluciones cuasi-óptimas), pero no tienen pruebas matemáticas que aseguren que la solución encontrada sea la mejor de todas.

3.3 Heurísticas y Metaheurísticas

El término *heurística* (del griego *heuriskein*, "encontrar" o "descubrir") hace referencia a estrategias que utilizan conocimiento específico del dominio del problema o "reglas prácticas" para guiar la búsqueda hacia soluciones de alta calidad. Si bien producen soluciones cuasi-óptimas de forma rápida, a menudo corren el riesgo de quedar atrapadas en óptimos locales (soluciones que son las mejores en su vecindario inmediato, pero no en todo el espacio de búsqueda).

Para superar esta limitación nacen las **metaheurísticas**. Una metaheurística se define como un marco algorítmico de nivel superior, independiente del problema subyacente, que provee un conjunto de directrices para guiar a otras heurísticas subordinadas en la exploración del espacio de soluciones.

El éxito de cualquier metaheurística reside en mantener un delicado equilibrio dinámico entre dos conceptos fundamentales:

- **Exploración (o diversificación):** La capacidad del algoritmo para investigar regiones globales y desconocidas del espacio de búsqueda, evitando la convergencia prematura.
- **Explotación (o intensificación):** La capacidad de concentrar la búsqueda en las vecindades de las mejores soluciones encontradas hasta el momento, con el fin de refinar y mejorar dichos resultados.

3.4 Clasificación de las Metaheurísticas

A lo largo de las últimas décadas, la literatura científica ha propuesto numerosas metaheurísticas, muchas de ellas inspiradas en procesos naturales, biológicos o físicos. La clasificación más extendida en la comunidad académica divide estos algoritmos según el número de soluciones con las que trabajan simultáneamente en cada iteración:

3.4.1 Metaheurísticas basadas en trayectoria

También conocidas como métodos de búsqueda puntual, estos algoritmos mantienen y modifican una única solución a lo largo del tiempo, describiendo una trayectoria a través del espacio de búsqueda. En cada iteración, la solución actual se reemplaza por otra proveniente de su vecindario si se cumple un determinado criterio de aceptación. Ejemplos clásicos de esta categoría son la Búsqueda Local, el Recocido Simulado (*Simulated Annealing*) y la Búsqueda Tabú.

3.4.2 Metaheurísticas basadas en población

A diferencia de los métodos de trayectoria, estas técnicas mantienen en memoria un conjunto (o población) de soluciones que evolucionan de manera simultánea en cada iteración. Esta característica les confiere una mayor robustez y una capacidad de exploración superior, ya que las soluciones pueden interactuar e intercambiar información entre sí. Dentro de este paradigma destacan los Algoritmos Evolutivos (como los Algoritmos Genéticos) y la Optimización por Enjambre de Partículas (PSO).

3.5 Extensión a la Optimización Multiobjetivo

En gran parte de las aplicaciones del mundo real, los problemas no presentan un único objetivo a optimizar, sino múltiples objetivos que a menudo se encuentran en conflicto directo (por ejemplo, maximizar el rendimiento de un componente a la par que se minimiza su coste de fabricación).

En estos escenarios, denominados Problemas de Optimización Multiobjetivo (MOP), el concepto de "solución óptima" desaparece para dar paso al concepto de *Dominancia de Pareto*. El objetivo de las metaheurísticas multiobjetivo no es encontrar un único punto, sino aproximar el Frente de Pareto: un conjunto de soluciones no dominadas que representan los mejores compromisos posibles entre los diferentes objetivos evaluados, permitiendo finalmente al tomador de decisiones elegir la alternativa que mejor se adapte a sus necesidades.

Especificación y diseño

El diseño y modelado de la arquitectura representan la fase de mayor complejidad y dedicación en el desarrollo. La integridad de esta estructura determina la calidad técnica, la eficiencia y la mantenibilidad de la biblioteca. Con el objetivo de garantizar un nivel de abstracción óptimo que permita su aplicación en diversos casos de uso, se han integrado los siguientes criterios de diseño:

Los límites del problema no se encuentran en el algoritmo, sino en el problema

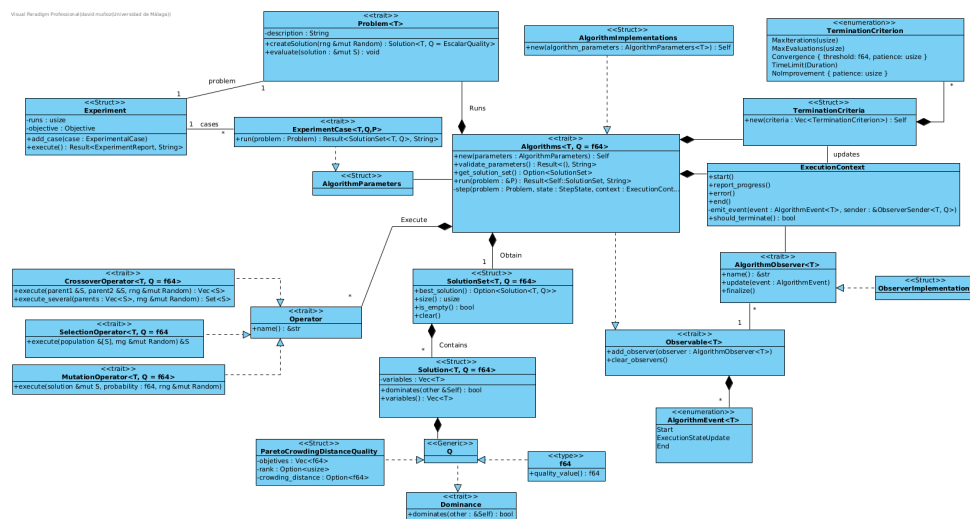


Figura 4.1. Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [10].

4.2 Módulos

A continuación, se describen los distintos módulos que componen la biblioteca y las responsabilidades asociadas a cada uno de ellos.

4.2.1 Algoritmos

Este módulo es el encargado de ejecutar los distintos problemas definidos en la biblioteca mediante la función `run`, la cual recibe como argumento una referencia al problema que se desea resolver.

Para su configuración e instanciación, cada algoritmo se apoya en una estructura de parámetros que actúa como su base configurativa. Esta estructura encapsula toda la información necesaria para la construcción del algoritmo y desempeña un papel crucial en el módulo de experimentos, donde se utiliza para almacenar la configuración específica de cada algoritmo que servirá como caso de prueba.

Con el objetivo de garantizar la integridad de la configuración antes de la ejecución, los algoritmos incorporan un sistema de verificación basado en la función `validate_parameters(&self) -> Result<(), String>`. Este método se invoca de manera sistemática antes de iniciar el proceso de resolución para validar que los parámetros introducidos sean coherentes y correctos. En caso de que la configuración no cumpla con los requisitos establecidos, la función devuelve un error con un mensaje personalizado, lo que permite al usuario identificar y corregir anomalías en la configuración de forma precisa.

Para llevar a cabo su funcionamiento, los algoritmos requieren la aplicación de *operadores*, responsables de realizar las transformaciones necesarias sobre las soluciones con el objetivo de explorarlas y mejorarlas de forma iterativa. Gracias a la arquitectura modular adoptada, dichos operadores son intercambiables, lo que permite adaptar el comportamiento del algoritmo sin alterar su implementación interna. De igual modo, esta modularización facilita la sustitución o modificación del problema sobre el que se ejecuta el algoritmo de manera sencilla y flexible.

Asimismo, los algoritmos admiten la incorporación directa de *observadores*, cuyo propósito es analizar y monitorizar su comportamiento durante la ejecución. Estos observadores se describen con mayor detalle en su apartado correspondiente.

Estos algoritmos pueden ejecutarse de forma concurrente o paralela, en función de la configuración seleccionada por el usuario. En este trabajo se ofrece la posibilidad de especificar manualmente el número de hilos a crear o, alternativamente, delegar esta decisión al propio programa. En este último caso, el número de hilos se determina automáticamente en función de las características del sistema en el que se ejecuta, haciendo uso de la función `available_parallelism()` proporcionada por la librería estándar.

4.2.2 Observadores

Los observadores estarán asociados a aquellos algoritmos que implementen el *trait Observable*. Dicho *trait* proporciona la funcionalidad necesaria para añadir y eliminar observadores.

Para la comunicación entre el algoritmo y los observadores adjuntados, se ha diseñado un sistema basado en canales (*channels*), un patrón idiomático en Rust para la comunicación segura entre hilos. Inicialmente, se planteó una implementación en la que el *trait Observable* incluyese un método encargado de notificar directamente a todos los observadores registrados. Sin embargo, debido a las restricciones impuestas por el sistema de propiedad y préstamos de Rust (*Borrow Checker*), esta aproximación resultó compleja de implementar de forma clara y segura.

Como alternativa, se optó por un modelo concurrente en el que el algoritmo de optimización se ejecuta en un hilo independiente, mientras que los observadores se mantienen a la espera en hilos separados, recibiendo de manera asíncrona la información enviada por el algoritmo a través de los canales. Este enfoque simplifica la gestión de la concurrencia, evita conflictos de préstamos y garantiza una comunicación segura entre los distintos componentes del sistema.

————— FALTA

Existen distintos tipos de observadores, entre los que se incluyen aquellos que muestran la información directamente por terminal o los que generan tablas y estructuras de datos más elaboradas. Estos observadores permiten analizar la evolución del proceso de optimización en función del número de evaluaciones realizadas, proporcionando al usuario una visión clara y detallada del comportamiento del algoritmo.

————— IDEAS

Los *ChartObservers*, generarán varios gráficos, uno que muestre como avanza la calidad de las soluciones junto a las evaluaciones, como avanzan las evaluaciones y uno con las soluciones en general. En el caso de que las soluciones sean un vector bidimensional, un gráfico de puntos, donde estos estén en las coordenadas de los objetivos, en el caso de que no sea un vector, pues un gráfico de barras, con las veces que se ha repetido cada valor y si fuera una vector de más de 2 dimensiones, podríamos generar varios gráficos con los diferentes casos. Se plantea generar un html con toda la información super currado.

—————

4.2.3 Operadores

Este componente constituye el núcleo de la biblioteca de optimización, ya que encapsula la lógica fundamental de los algoritmos implementados. En particular, se definen tres tipos principales de operadores, cada uno de los cuales desempeña un papel esencial dentro del proceso de optimización:

- **Operadores de mutación:** introducen variaciones aleatorias en los individuos de la población con el objetivo de mantener la diversidad genética y evitar la convergencia prematura hacia soluciones subóptimas. Estos operadores permiten explorar nuevas regiones del espacio de búsqueda mediante modificaciones controladas de las soluciones existentes.
- **Operadores de selección:** se encargan de elegir los individuos que participarán en la generación de nuevas soluciones, generalmente en función de su calidad o valor de aptitud. Su función principal es favorecer la propagación de las mejores soluciones encontradas hasta el momento, equilibrando la presión selectiva con la preservación de la diversidad poblacional.
- **Operadores de cruce:** combinan la información de dos o más individuos seleccionados para generar nuevos descendientes. A través de este proceso se busca explotar las características favorables de las soluciones existentes, recombinándolas de manera que puedan dar lugar a individuos con un mejor desempeño.

4.2.4 Problemas

Este módulo contiene la información del problema que se quiere resolver. El "trait" que deben implementar todos los problemas es muy sencillo; solo obliga a crear unas simples funciones para obtener y editar la descripción del problema en sí, otras funciones para evaluar las soluciones al problema, y otra para crear una solución de la que partirá el algoritmo que intente resolver el problema. — Quizás también deben implementar otra función para elegir la mejor solución para el problema —

Los problemas pueden ser muy diversos, por tanto no es fácilmente delimitable este módulo. Para ayudar a diseñar problemas en esta biblioteca, se muestran ejemplos de estos problemas e implementaciones para ayudar a la construcción de estos con el patrón "Builder".

4.2.5 Conjuntos de soluciones

Los algoritmos devolverán esta estructura de datos; es un "wrapper" que contiene una lista con las soluciones obtenidas por el algoritmo al intentar solucionar el problema.

4.2.6 Soluciones

Las soluciones representan las estructuras que almacenan el resultado generado por la aplicación iterativa de los operadores dentro de los algoritmos de optimización.

En una primera aproximación del diseño, `Solution` se definió como un *trait* que declaraba un conjunto de funciones fundamentales, tales como la obtención del valor de *fitness*, el acceso a las variables de la solución o la recuperación de su representación interna. Este *trait* debía ser implementado por distintos *structs*, cada uno adaptado a las necesidades específicas de cada problema.

Por ejemplo, si el problema requería representar una solución como una permutación binaria, el *struct* correspondiente contendría como atributos un vector binario junto con el valor de calidad asociado a dicha solución. Sin embargo, se observó que esta arquitectura introducía una elevada fragmentación en las implementaciones y dificultaba la generalización de la biblioteca para soportar un mayor número de problemas y algoritmos.

Por este motivo, se optó por una solución más abstracta y flexible. En el diseño actual, `Solution` se define como un *struct* genérico parametrizado por dos tipos: T y Q. El tipo T representa el tipo de dato de las variables de decisión almacenadas en la solución, mientras que Q representa el tipo asociado a la evaluación de calidad de dicha solución.

Para poder comparar soluciones entre sí, el tipo Q debe implementar un *trait* personalizado denominado `Dominance`. Este *trait* define el comportamiento necesario para establecer relaciones de dominancia entre valores de calidad, proporcionando un método que permite comparar dos instancias del mismo tipo y determinar cuál de ellas representa una solución superior.

Adicionalmente, la estructura `Solution` expone un método denominado `dominates`, cuya implementación delega la comparación en el método definido por el *trait* `Dominance`. De esta manera, la lógica de comparación queda desacoplada de la representación de la solución, permitiendo soportar tanto optimización monoobjetivo como multiobjetivo mediante diferentes implementaciones del tipo Q.

4.2.7 Experimentos

El módulo de experimentos constituye la capa de abstracción superior de la biblioteca, diseñada para facilitar la evaluación sistemática y comparativa de los diversos algoritmos frente a un problema específico. Su propósito fundamental es permitir que el usuario final realice un proceso de *benchmarking* robusto, identificando qué configuraciones ofrecen un rendimiento óptimo mediante la ejecución controlada y automatizada de baterías de pruebas.

Este servicio se articula a través de la definición de casos de prueba, los cuales representan

combinaciones específicas de parámetros y componentes. Gracias a la arquitectura modular de `rMetal`, el sistema permite explorar un amplio espacio de configuraciones, incluyendo la variación de operadores, el ajuste de las probabilidades de mutación y cruce, o la selección de diferentes criterios de terminación. Asimismo, es posible comparar directamente algoritmos distintos bajo las mismas condiciones de contorno, lo que proporciona una base empírica sólida para la toma de decisiones técnicas.

Al finalizar las ejecuciones, el módulo genera un reporte detallado que sintetiza los resultados obtenidos por cada configuración. Este informe no solo recopila las soluciones halladas, sino que ofrece una visión analítica del comportamiento de los algoritmos, permitiendo al usuario discernir con criterio qué estrategia es la más adecuada para la resolución de su problema.

4.2.8 Utilidades

Para posibilitar el desarrollo de este proyecto y cumplir el objetivo de mantener una arquitectura con *cero dependencias externas*, se han implementado pequeños módulos auxiliares utilizados en el núcleo de la biblioteca. Estos módulos reproducen de forma ligera y controlada parte de la funcionalidad ofrecida por `crates` ampliamente utilizados en el ecosistema Rust, como `rand` [3] o `plotters` [5]. De este modo, se evita la inclusión de bibliotecas externas sin renunciar a las capacidades necesarias para el correcto funcionamiento del sistema.

5

Implementación y pruebas

Este capítulo detalla el proceso de construcción de la biblioteca `rMetal`, trasladando los conceptos teóricos y el diseño arquitectónico descritos en los capítulos anteriores a código real en el lenguaje de programación Rust.

5.1 Implementación

Una vez definida y consolidada la arquitectura de la biblioteca, se inició la fase de construcción modular siguiendo un enfoque ascendente (*bottom-up*). Contrariamente a lo que se podría suponer inicialmente, situando el foco en los algoritmos o en la definición de los problemas, el desarrollo comenzó por el módulo de las **soluciones**. Esta decisión estratégica se fundamenta en que la estructura que define a la solución es, en realidad, el componente más determinante del sistema, actuando como el eje central sobre el que pivotan todos los demás módulos.

La importancia de este módulo radica en que la solución es la entidad que los operadores transforman para explorar el espacio de búsqueda y la que los observadores monitorizan constantemente para extraer métricas de rendimiento. Del mismo modo, los problemas dependen de esta estructura para realizar tanto la generación inicial de individuos como su posterior evaluación, mientras que los algoritmos actúan como gestores encargados de almacenarlas y orquestar su flujo a lo largo de las iteraciones.

Debido a esta interdependencia, un diseño deficiente o una implementación poco optimizada de esta estructura central comprometería no solo la **extensibilidad** de la biblioteca —al dificultar la integración de nuevos problemas o tipos de datos— sino también su **rendimiento computacional**.

Dado que la solución es el objeto con mayor frecuencia de acceso y modificación, cualquier ineficiencia en su desarrollo se convertiría inevitablemente en un cuello de botella durante ejecuciones intensivas. Por ello, garantizar una base técnica sólida en este módulo fue la prioridad absoluta antes de proceder con la implementación del resto de componentes.

Listing 1 Estructura desarrollada para las Soluciones

```
1  #[derive(Clone, Debug)]
2  pub struct Solution<T, Q = f64> {
3      pub variables: Vec<T>,
4      pub quality: Option<Q>,
5  }
```

Posteriormente se abordó la implementación de los **problemas**, definidos mediante un **trait** que actúa como el contrato de interfaz para cualquier modelo de optimización que se desee resolver. Para garantizar que la biblioteca fuera lo suficientemente flexible como para soportar diversos dominios de problemas —desde optimización continua hasta combinatoria—, se diseñó una estructura altamente genérica:

Listing 2 Definición genérica del **trait Problem** para la abstracción de problemas de optimización.

```
1  pub trait Problem<T, Q = f64>
2  where
3      T: Clone,
4      Q: Clone + Default,
5  {
6      fn new() -> Self;
7
8      fn evaluate(&self, solution: &mut Solution<T, Q>);
9
10     fn create_solution(&self, _rng: &mut Random) -> Solution<T, Q>;
11
12     fn set_problem_description(&mut self, description: String);
13
14     fn get_problem_description(&self) -> String;
15
16     fn get_improvement_direction(&self) -> ImprovementDirection;
17 }
```

Como se observa en la definición, el uso de parámetros genéricos permite desacoplar la lógica del problema de la representación interna de los datos. El parámetro **T** representa el tipo de variable de decisión, mientras que **Q** define el tipo de la evaluación (u objetivo), que por defecto se establece

como `f64`. Esta abstracción delega en el usuario la responsabilidad de definir aspectos críticos como la dirección de mejora (`ImprovementDirection`), permitiendo configurar si el objetivo es la maximización o minimización.

Asimismo, el `trait` estandariza la forma en la que los algoritmos interactúan con el dominio del problema: la función `evaluate` permite calificar la calidad de una solución, mientras que `create_solution` define el mecanismo de generación inicial. Esta estructura asegura que los algoritmos puedan operar de forma agnóstica al problema concreto, facilitando la extensibilidad del sistema. Una vez establecida esta base, el desarrollo prosiguió con la implementación de los operadores y los algoritmos, encargados de explotar estas definiciones para hallar soluciones óptimas.

5.2 Desarrollo de pruebas

La utilización de un correcto conjunto de pruebas agiliza en gran medida el trabajo de programación en cualquier fase del desarrollo. La utilización de un correcto conjunto de pruebas agiliza en gran medida el trabajo de programación en cualquier fase del desarrollo, previniendo regresiones y garantizando la estabilidad del código. Poder verificar de forma determinista que el código subyacente opera de manera correcta resulta indispensable. Para llevar a cabo esta tarea, el proyecto se ha apoyado fuertemente en el *framework* de pruebas integrado de forma nativa en el ecosistema de Rust (`cargo test`).

5.2.1 Pruebas unitarias

Dada la metodología de desarrollo ascendente adoptada, la implementación de pruebas unitarias se convirtió en un pilar fundamental para asegurar la robustez de cada componente antes de escalar hacia niveles de abstracción superiores. El objetivo principal de estas pruebas fue verificar de forma aislada e independiente el comportamiento de las funciones y estructuras mínimas de cada módulo, garantizando que cada unidad de código cumpliera estrictamente con su responsabilidad única.

En consonancia con las prácticas estándar del ecosistema Rust, estas pruebas unitarias se encuentran integradas directamente en el mismo archivo que el código fuente del módulo que evalúan. Generalmente ubicadas al final de cada archivo bajo un módulo específico anotado con el atributo `#[cfg(test)]`, esta disposición permite mantener una relación estrecha entre la implementación y su verificación. Además, esta estructura facilita el acceso a elementos privados del módulo, permitiendo una validación exhaustiva de la lógica interna que no sería accesible desde el exterior, garantizando así que cada componente sea intrínsecamente correcto antes de ser expuesto.

Este enfoque de aislamiento resultó especialmente crítico en el desarrollo de la estructura de los operadores, donde pequeñas desviaciones en la lógica podrían propagar errores difíciles de detectar en

fases posteriores. Al diseñar pruebas simples y focalizadas en una única funcionalidad, se facilitó la identificación inmediata de regresiones durante el proceso de refactorización. Además, estas pruebas permitieron validar la gestión de tipos y la integridad de los datos en tiempo de ejecución, asegurando que los cimientos de la biblioteca fueran sólidos antes de proceder a la integración de sistemas más complejos.

Listing 3 Prueba unitaria que prueba el correcto funcionamiento de la función `dominance` en rangos diferentes para `pareto_crowding_solution.rs`.

```
1  #[test]
2  fn dominates_prefers_lower_rank() {
3      let a = MultiObjectiveRealSolutionBuilder::from_variables(vec![0.0, 0.0])
4          .with_rank(0)
5          .with_crowding_distance(0.1)
6          .build();
7      let b = MultiObjectiveRealSolutionBuilder::from_variables(vec![0.0, 0.0])
8          .with_rank(1)
9          .with_crowding_distance(0.1)
10         .build();
11
12     assert!(a.dominates(&b));
13     assert!(!b.dominates(&a));
14 }
```

5.2.2 Pruebas de integración

Una vez validada la fiabilidad de los componentes individuales mediante el testeo unitario, se procedió al desarrollo de las pruebas de integración. Siguiendo las convenciones de estructuración de proyectos en Rust, este conjunto de pruebas se localiza de forma independiente en el directorio `tests/`, situado en la raíz del proyecto `rMetal`. Esta separación física respecto al código fuente alojado en `src/` es fundamental, ya que permite tratar a la biblioteca como una entidad externa, evaluando exclusivamente su interfaz pública y la interacción entre sus diversos módulos.

Mientras que las pruebas previas evalúan piezas aisladas, estas se han diseñado con el propósito de comprobar que todos los componentes de `rMetal` interactúan armónicamente entre sí para resolver un problema de optimización de principio a fin. El foco de estas pruebas reside en la orquestación del flujo de trabajo: desde la configuración e instanciación de un algoritmo, pasando por la validación sistemática de sus parámetros, hasta la ejecución iterativa donde intervienen operadores, problemas y observadores en un entorno compartido.

Estas pruebas de mayor nivel permiten verificar que la comunicación entre módulos —por ejemplo,

el intercambio de soluciones entre el algoritmo y el problema— se realiza de forma eficiente y sin errores de concordancia. Al ejecutarse desde la raíz del proyecto, estas pruebas simulan el uso real que tendría un usuario final de la biblioteca. En última instancia, las pruebas de integración garantizan que el sistema completo es capaz de cumplir con su propósito funcional, ofreciendo una visión global de la estabilidad y el rendimiento de la biblioteca ante casos de uso reales.

Listing 4 Prueba de integración sobre un ejemplo de error donde el tamaño de la población se inicializa a cero.

```
1      #[test]
2      fn ga_returns_error_with_invalid_parameters_edge_case() {
3          // Edge case: invalid parameter configuration should return a validation error.
4          let problem = KnapsackBuilder::new()
5              .with_capacity(10.0)
6              .add_item(5.0, 10.0)
7              .build();
8
9          let parameters = GeneticAlgorithmParameters::new(
10             0,
11             0.8,
12             0.1,
13             SinglePointCrossover::new(),
14             BitFlipMutation::new(),
15             BinaryTournamentSelection::new(),
16             TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(5)]),
17         )
18         .with_seed(1);
19
20         let mut algorithm = GeneticAlgorithm::new(parameters);
21         let error = match algorithm.run(&problem) {
22             Ok(_) => panic!("GA must return an error for invalid parameters"),
23             Err(message) => message,
24         };
25         assert!(error.contains("population_size"));
26     }
```

6

Validación experimental

En este capítulo se presenta la validación práctica de `rMetal` mediante una serie de experimentos diseñados para evaluar tanto la funcionalidad como la flexibilidad de la biblioteca. Estas pruebas se han integrado en el sistema nativo de ejemplos de `Cargo`, lo que permite que cualquier usuario pueda reproducir los resultados de forma directa. Para ejecutar estos escenarios de prueba, se utiliza el comando estándar del ecosistema Rust: `cargo run --example <nombre_ejemplo>`.

El primero de estos escenarios es el denominado `mono_objective_experiment`. Este experimento tiene como objetivo resolver una instancia del problema de la mochila (*Knapsack Problem*) comparando la eficacia de diversas estrategias algorítmicas bajo condiciones controladas.

Listing 5 Implementación del experimento mono-objetivo (mono_objective_experiment) utilizando el problema de la mochila.

```

1  use rmetal::algorithms::{
2      GeneticAlgorithmExperiment, GeneticAlgorithmParameters, HillClimbingParameters,
3      PSOParameters, SimulatedAnnealingParameters, TerminationCriteria, TerminationCriterion,
4  };
5  use rmetal::experiment::{Experiment, Objective};
6  use rmetal::operator::{BinaryTournamentSelection, BitFlipMutation, SinglePointCrossover};
7  use rmetal::problem::KnapsackBuilder;
8
9  fn main() {
10     // Definición de la instancia del problema de la mochila
11     let problem = KnapsackBuilder::new()
12         .with_capacity(150.0)
13         .add_item(1.0, 2.0)
14         .add_item(45.0, 100.0)
15         .add_item(90.0, 301.0)
16         .add_item(150.0, 301.0) // ... (resto de items)
17         .build();
18
19     // Configuración de los casos de prueba (Algoritmos)
20     let hill_climbing_case = HillClimbingParameters::new(
21         BitFlipMutation::new(),
22         0.12,
23         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(180)]),
24     );
25
26     let genetic_algorithm_case = GeneticAlgorithmExperiment::new(
27         GeneticAlgorithmParameters::new(
28             80, 0.90, 0.06,
29             SinglePointCrossover::new(),
30             BitFlipMutation::new(),
31             BinaryTournamentSelection::new(),
32             TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(60)]),
33         )
34         .with_elite_size(1)
35         .with_threads(4),
36     );
37
38     let simulated_annealing_case = SimulatedAnnealingParameters::new(
39         BitFlipMutation::new(), 0.10, 45.0, 0.985,
40         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(220)]),
41     ).with_seed(777);
42
43     let pso_case = PSOParameters::new(
44         50, 0.72, 1.49, 1.49,
45         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(120)]),
46     ).with_velocity_clamp(4.0).with_seed(999);
47
48     // Orquestación y ejecución del experimento
49     let report = Experiment::new(problem)
50         .with_runs(24)
51         .with_objective(Objective::Maximize)
52         .add_case(hill_climbing_case)
53         .add_case(genetic_algorithm_case)
54         .add_case(simulated_annealing_case)
55         .add_case(pso_case)
56         .execute();
57
58     match report {
59         Ok(report) => println!("{}", report.to_text_table()),
60         Err(error) => eprintln!("Experiment execution failed: {}", error),
61     }
62 }

```

Como se observa en la implementación, el experimento comienza con la construcción de una instancia del problema mediante `KnapsackBuilder`, definiendo una capacidad máxima y un conjunto de ítems con sus respectivos pesos y valores. Posteriormente, se definen cuatro configuraciones algorítmicas distintas: *Hill Climbing*, Algoritmo Genético, *Simulated Annealing* y Optimización por Enjambre de Partículas (PSO). Cada uno de estos casos permite un ajuste fino de sus parámetros específicos, como el tamaño de la población, las probabilidades de mutación o el uso de semillas (*seeds*) para garantizar la reproducibilidad de los resultados.

Resulta relevante destacar la capacidad de personalización que ofrece la biblioteca; por ejemplo, en el Algoritmo Genético se especifica el uso de paralelismo mediante la creación de 4 hilos de ejecución. Una vez configurados, todos los casos se añaden a una instancia de `Experiment`, la cual se encarga de orquestar 24 ejecuciones por cada algoritmo para asegurar significancia estadística. Finalmente, el sistema genera un reporte detallado en formato de tabla, permitiendo comparar de forma directa el rendimiento y la convergencia de cada estrategia frente al objetivo de maximización propuesto.

7

Conclusiones y líneas futuras

———— Problemas ————

No determino entre máquinas distintas en ejecuciones paralelas, no se si es un problema, pero si una máquina tiene un procesador con más núcleos que otra máquina, al ejecutar paralelamente un algoritmo con la misma semilla, puede dar lugar a ejecuciones distintas debido a que el algoritmo generará más hilos concurrentes por los cuales puede llegar a conclusiones distintas. (Creo que es importante recalcarlo) —

El desarrollo de este software ha representado un cambio de paradigma respecto a mi formación académica previa. La complejidad técnica no radica meramente en la sintaxis, sino en la comprensión profunda del ecosistema del lenguaje, así como de sus virtudes y limitaciones intrínsecas.

Rust supuso un reto, un lenguaje de sistemas que, si bien permite ciertos patrones de diseño, se aleja de la Programación Orientada a Objetos convencional de lenguajes como Java a la cual estaba acostumbrado. En su lugar, propone un modelo basado en estructuras (structs) y rasgos (traits), estos últimos con una clara influencia de las clases de tipos de Haskell.

Estas variaciones desembocaron en problemas durante el desarrollo de la arquitectura, al no existir tampoco herencia, un factor limitante en el desarrollo de la arquitectura

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover**, de **JetBrains**,

Apéndice A. Installation

Manual

En esta sección se describe el proceso de instalación y puesta en marcha de la biblioteca de optimización metaheurística desarrollada en este trabajo. Dado que el proyecto ha sido implementado íntegramente en Rust, su integración en otros proyectos resulta sencilla y se ajusta a los mecanismos estándar proporcionados por el propio ecosistema del lenguaje.

A.1 Requisitos

Para poder utilizar la biblioteca es necesario disponer de los siguientes elementos:

- Un sistema operativo compatible con el compilador de Rust (Linux, macOS o Windows).
- El compilador de Rust y su gestor de paquetes **Cargo**, instalados y correctamente configurados.
- Acceso a Internet para la descarga del código fuente o del *crate* correspondiente.

La instalación del entorno de desarrollo de Rust puede realizarse siguiendo las instrucciones oficiales proporcionadas por el lenguaje.

A.2 Instalación desde el repositorio

El método más directo para utilizar la biblioteca consiste en clonar el repositorio desde la plataforma GitHub y compilar el proyecto de forma local. Este enfoque resulta especialmente útil para desarrolladores que deseen inspeccionar, modificar o extender el código fuente.

El repositorio puede descargarse ejecutando el siguiente comando:

```
git clone https://github.com/DRLKs/rMetal
```

Una vez clonado el repositorio, basta con acceder al directorio del proyecto y ejecutar:

```
cargo build
```

Este comando descargará automáticamente las dependencias necesarias y generará la biblioteca en modo desarrollo. Para ejecutar ejemplos incluidos en el repositorio o realizar pruebas, puede utilizarse el comando:

```
cargo run
```

A.3 Instalación como *crate* mediante Cargo

Dado que la biblioteca ha sido diseñada siguiendo las convenciones estándar del ecosistema Rust, también puede integrarse fácilmente como dependencia en cualquier proyecto Rust mediante el gestor de paquetes **Cargo**. Este método es el más recomendado para usuarios que deseen emplear la biblioteca como componente dentro de una aplicación mayor.

Para ello, basta con añadir la dependencia correspondiente en el archivo **Cargo.toml** del proyecto:

```
[dependencies]
rMetal = "0.1.0"
```

Una vez añadida la dependencia, Cargo se encargará automáticamente de descargar, compilar e integrar la biblioteca durante el proceso de construcción del proyecto. A partir de ese momento, la funcionalidad proporcionada por la biblioteca podrá utilizarse directamente desde el código fuente mediante las instrucciones `use crate::` habituales del lenguaje.

Este mecanismo de distribución facilita la reutilización del software desarrollado y permite mantener actualizada la biblioteca de forma sencilla a medida que se publiquen nuevas versiones.

Bibliografía

- [1] Atlassian. *JIRA: Issue and Project Tracking Software*. Version 9.0. Herramienta para gestión de proyectos, seguimiento de incidencias y colaboración en equipo. URL: <https://www.atlassian.com/software/jira> (visited on 02/17/2026).
- [2] The rust-analyzer authors. *rust-analyzer*. URL: <https://github.com/rust-lang/rust-analyzer> (visited on 02/09/2026).
- [3] The Rand Project Developers and The Rust Project Developers. *rand: Rust random number generators and utilities*. Version 0.10.0. Crate for random number generation in Rust (MIT OR Apache-2.0). URL: <https://github.com/rust-random/rand> (visited on 02/17/2026).
- [4] J. J. Durillo and A. J. Nebro. *jMetalPy*. URL: <https://github.com/jMetal/jMetalPy> (visited on 02/09/2026).
- [5] Hao Hou, Aaron Erhardt, and contributors. *Plotters: A Rust drawing and plotting library*. Version 0.3.7. Librería Rust para generar gráficos y visualizaciones de datos. URL: <https://github.com/plotters-rs/plotters> (visited on 02/17/2026).
- [6] A. J. Nebro J. J. Durillo. *jMetal*. URL: <https://github.com/jMetal/jMetal> (visited on 02/09/2026).
- [7] Thorsten Leemhuis. *Linux Kernel: Rust Support Officially Approved*. heise online. Dec. 2025. URL: <https://www.heise.de/en/news/Linux-Kernel-Rust-Support-Officially-Approved-11109808.html> (visited on 02/15/2026).
- [8] Stack Overflow. *Stack Overflow Developer Survey 2025*. URL: <https://survey.stackoverflow.co/2025/> (visited on 10/12/2025).
- [9] David Muñoz del Valle. *PokerHelper*. URL: <https://github.com/DRLKs/PokerHelper> (visited on 02/09/2026).
- [10] Visual Paradigm International. *Visual Paradigm Professional*. Version 17.0. Herramienta profesional para modelado UML, diseño de arquitectura software y gestión del ciclo de vida del desarrollo. URL: <https://www.visual-paradigm.com> (visited on 02/17/2026).

- [11] Xin-She Yang and colleagues. *MEALPY: MEta-heuristic ALgorithms in PYthon*. URL: <https://mealpy.readthedocs.io/en/latest/pages/general/introduction.html> (visited on 02/09/2026).



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga